SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	**Assessment:**	Constraint Based Problem Solving
Weightage:	40%	**Total Marks:**	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Kolar Umme Sadiya	**Reg. No.:**	192425117
Section / Batch:	3rd year /AI&ML	**Date:**	26/08/2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Design an I/O subsystem for a real-time robotic arm controller with the following constraints:
   - Position sensors generate data every 500 µs.
   - Control commands must be processed within 5 µs.
   - Use priority-based interrupt handling for critical sensors.
   - Non-critical diagnostic data must be transferred using DMA.
2. Construct a communication architecture for an autonomous vehicle with the following constraints:
   - Multiple cameras generate a combined data rate of 2 GB/s.
   - Safety-critical sensors require deterministic response time.
   - Use PCIe for high-bandwidth devices.
   - Ensure that low-priority devices do not affect safety-critical communication.
3. Design an I/O mechanism for an industrial monitoring system with the following constraints:
   - 100 sensors continuously generate measurement data.
   - The system must support real-time fault detection.
   - Use interrupt coalescing to reduce processor overhead.
   - Critical fault signals must receive immediate interrupt service.
4. Construct a high-speed network interface subsystem with the following constraints:
   - Network traffic can reach 10 Gb/s.
   - The CPU must be protected from excessive packet-processing overhead.
   - Use DMA-based packet transfer between the network interface and memory.
   - Generate interrupts only after processing a configurable batch of packets.
5. Design an I/O architecture for a smart healthcare monitoring system with the following constraints:
   - Patient-monitoring devices generate data at different sampling rates.
   - Emergency signals must be handled with highest priority.
   - Use nested interrupt handling for urgent events.
   - Store continuous monitoring data using DMA transfers without interrupting critical processing.

Code of Conduct

I Kolar Umme Sadiya-192425117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kolar Umme Sadiya

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator **Faculty Incharge**

1. Design an I/O subsystem for a real-time robotic arm controller

Problem Understanding

A real-time robotic arm controller receives position information from sensors and must respond to control requirements within a very short time. The system has the following constraints:

- Position sensors generate data every 500 μs.
- Control commands must be processed within 5 μs.
- Critical sensors require priority-based interrupt handling.
- Non-critical diagnostic data must be transferred using DMA.

The main objective is to provide fast and deterministic processing for critical sensor data while reducing CPU overhead for non-critical data.

Constraint Analysis

The sensor generates data every 500 μs.

Sensor data rate:

1 / 500 μs = 2000 sensor updates per second

The most important constraint is the **5 μs control-command processing time**. Therefore, critical sensor interrupts must receive the highest priority.

Diagnostic data is not time-critical, so using interrupts for every diagnostic transfer would unnecessarily increase CPU overhead. DMA is more suitable for this data.

Proposed I/O Architecture

The proposed architecture is:

Position Sensors → Interrupt Controller → CPU → Motor Controller

Diagnostic Devices → DMA Controller → Main Memory

The position sensors are connected to a priority-based interrupt controller. When a critical sensor generates data, an interrupt is immediately sent to the CPU.

The interrupt controller assigns the highest priority to critical position sensors. The CPU temporarily handles the critical sensor event and generates the required control command.

For diagnostic information, the DMA controller transfers data directly between the diagnostic device and main memory without requiring the CPU to handle every data word.

Working

1. Position sensors continuously generate data every 500 µs.
2. A critical sensor generates an interrupt when new position data is available.
3. The interrupt controller identifies the sensor priority.
4. The highest-priority interrupt is sent to the CPU.
5. The CPU processes the sensor data and generates the required control command.
6. The control command must complete within the specified 5 µs requirement.
7. Diagnostic devices send non-critical information to the DMA controller.
8. DMA transfers diagnostic data directly to memory.
9. The CPU is notified only when necessary, such as after a DMA transfer is completed.

Why This Solution Is Suitable

Priority-based interrupts ensure that critical position information is processed before non-critical activities. DMA reduces CPU involvement in diagnostic data transfers.

This architecture provides low interrupt latency, efficient CPU utilization, and deterministic response for the robotic arm.

Conclusion

A priority-based interrupt system should be used for critical position sensors, while DMA should be used for non-critical diagnostic data. This combination satisfies the 500 µs sensor interval and supports the required 5 µs control response while minimizing CPU overhead.

2. Construct a communication architecture for an autonomous vehicle

Problem Understanding

An autonomous vehicle receives data from multiple cameras and safety-critical sensors. The system must process large amounts of data while ensuring that safety-related communication receives deterministic response time.

The given constraints are:

- Multiple cameras generate a combined data rate of 2 GB/s.
- Safety-critical sensors require deterministic response time.
- PCIe must be used for high-bandwidth devices.
- Low-priority devices must not affect safety-critical communication.

Constraint Analysis

The camera data rate is:

2 GB/s = 16 Gb/s

This is a very high data rate, so the communication interface must provide high bandwidth and efficient data transfer.

PCIe is suitable for high-bandwidth devices such as camera-processing hardware and accelerators.

Safety-critical sensors must have higher priority than normal devices. Therefore, the communication architecture should separate safety-critical traffic from low-priority traffic.

Proposed Architecture

The proposed architecture is:

Cameras → PCIe Interface → DMA → Main Memory

Safety Sensors → Priority Interrupt Controller → CPU

Low-Priority Devices → I/O Controller → DMA/Memory

The cameras are connected through PCIe because they generate a combined 2 GB/s data stream.

DMA can be used to transfer camera data directly into memory, reducing CPU involvement.

Safety-critical sensors are connected to a priority interrupt mechanism. Their interrupts are assigned the highest priority.

Low-priority devices are placed on separate communication paths or are assigned lower priority so that they cannot delay safety-critical communication.

Working

1. Cameras continuously generate approximately 2 GB/s of data.
2. Camera data is transferred through PCIe.
3. DMA transfers the camera data directly into main memory.
4. The CPU processes the required camera information.
5. Safety sensors generate interrupts when important events occur.
6. Safety-related interrupts receive the highest priority.
7. The CPU immediately processes safety-critical information.
8. Low-priority devices use the remaining communication resources.
9. Priority-based arbitration prevents low-priority devices from blocking safety-critical communication.

Why PCIe Is Suitable

PCIe provides high bandwidth and is suitable for high-speed devices such as cameras and accelerators. DMA further reduces CPU overhead by allowing direct data transfer between devices and memory.

Why Priority Is Important

Autonomous vehicles require fast responses to safety events. For example, a collision sensor or emergency braking sensor must receive faster service than a non-critical diagnostic device.

Conclusion

PCIe with DMA should be used for high-bandwidth camera data, while priority-based interrupt handling should be used for safety-critical sensors. Separating high-priority and low-priority traffic ensures deterministic response and prevents low-priority devices from affecting vehicle safety.

3. Design an I/O mechanism for an industrial monitoring system

Problem Understanding

An industrial monitoring system continuously receives data from 100 sensors. It must detect faults in real time while avoiding excessive CPU overhead.

The constraints are:

- 100 sensors continuously generate measurement data.
- Real-time fault detection is required.
- Interrupt coalescing must be used to reduce processor overhead.
- Critical fault signals must receive immediate interrupt service.

Constraint Analysis

Handling 100 sensors individually using an interrupt for every measurement could generate a large number of CPU interrupts.

This can increase CPU overhead and reduce the time available for actual processing.

Therefore, normal sensor data should use interrupt coalescing, while critical fault signals should bypass coalescing and generate immediate interrupts.

Proposed I/O Architecture

The architecture is:

100 Sensors → I/O Controller → Buffer → DMA/Memory

Critical Fault Signal → High-Priority Interrupt → CPU

Normal Sensor Data → Interrupt Coalescing → CPU

Sensor data can be collected into a buffer. Instead of generating an interrupt for every sensor measurement, the system waits until a configured number of events or a configured time interval is reached.

Working

1. The 100 sensors continuously generate measurement data.
2. Normal measurements are collected in a buffer.
3. DMA can transfer collected sensor data into memory.
4. Interrupt coalescing groups multiple normal events together.
5. The CPU receives one interrupt for a batch instead of one interrupt for every measurement.
6. If a critical fault occurs, the system immediately generates a high-priority interrupt.
7. The CPU stops lower-priority processing and handles the fault.
8. After the critical event is handled, normal monitoring continues.

Example

Suppose 20 normal sensor events are collected before generating an interrupt.

Instead of:

20 sensor events → 20 CPU interrupts

the system can use:

20 sensor events → 1 CPU interrupt

This significantly reduces processor overhead.

However, critical events should not wait for the batch. They should generate an immediate interrupt.

Why This Solution Is Suitable

Interrupt coalescing improves CPU efficiency by reducing the number of interrupts. At the same time, immediate interrupts for critical faults ensure that real-time fault detection is not delayed.

Conclusion

The proposed system combines interrupt coalescing for normal sensor measurements with high-priority immediate interrupts for critical faults. This provides efficient CPU utilization while maintaining real-time fault response.

4. Construct a high-speed network interface subsystem

Problem Understanding

The system must handle network traffic reaching 10 Gb/s while protecting the CPU from excessive packet-processing overhead.

The constraints are:

- Network traffic can reach 10 Gb/s.
- CPU overhead must be minimized.
- DMA must be used for packet transfer.
- Interrupts should be generated only after processing a configurable batch of packets.

Constraint Analysis

The maximum network data rate is:

10 Gb/s = 1.25 GB/s

At this data rate, generating an interrupt for every packet could cause excessive CPU overhead.

Therefore, DMA should transfer packets directly between the network interface and memory.

Interrupt coalescing should be used so that the CPU receives an interrupt only after a configured number of packets have been processed.

Proposed Architecture

The architecture is:

Network → Network Interface Card → DMA Controller → Main Memory → CPU

The network interface receives incoming packets and stores them in buffers.

The DMA controller transfers packet data directly to memory without requiring the CPU to copy each packet.

Working

1. Network traffic arrives at up to 10 Gb/s.
2. The network interface receives packets.
3. Packet data is placed into DMA buffers.
4. DMA transfers packets directly into main memory.
5. The CPU does not handle every individual data transfer.
6. Packets are processed in configurable batches.
7. After a batch is completed, the network interface generates an interrupt.
8. The CPU processes the batch of packets.
9. The process continues for subsequent batches.

Example

Assume the interrupt batch size is configured to 32 packets.

Instead of:

32 packets → 32 interrupts

the system can use:

32 packets → 1 interrupt

This greatly reduces CPU interrupt overhead.

The batch size can be adjusted according to system requirements. A smaller batch provides lower response latency, while a larger batch reduces interrupt overhead.

Advantages

- Reduces CPU overhead.
- Supports high-speed network traffic.
- Provides efficient memory transfer.
- Reduces the number of interrupts.
- Improves overall throughput.
- Allows the CPU to spend more time on application processing.

Trade-off

Very large batches reduce CPU interrupt overhead but can increase packet-processing latency. Very small batches reduce latency but increase the number of interrupts.

Therefore, the batch size should be configured according to the required balance between latency and CPU utilization.

Conclusion

A DMA-based network interface with interrupt coalescing is suitable for 10 Gb/s network traffic. DMA handles high-speed packet movement, while configurable interrupt batching protects the CPU from excessive interrupt overhead.

5. Design an I/O architecture for a smart healthcare monitoring system

Problem Understanding

A smart healthcare monitoring system receives data from multiple patient-monitoring devices. Different devices operate at different sampling rates, while emergency events require immediate processing.

The constraints are:

- Patient-monitoring devices generate data at different sampling rates.
- Emergency signals must have the highest priority.
- Nested interrupt handling must be used for urgent events.
- Continuous monitoring data must be transferred using DMA without interrupting critical processing.

Constraint Analysis

Different healthcare devices may generate data at different frequencies. For example, a continuous monitoring device may produce frequent measurements, while another device may produce data less frequently.

Emergency events such as critical patient conditions must be handled immediately.

If the CPU handles every normal monitoring data transfer through interrupts, CPU overhead can become high. Therefore, DMA should be used for continuous monitoring data.

Nested interrupts allow a high-priority emergency interrupt to interrupt the processing of a lower-priority event.

Proposed Architecture

The architecture is:

Patient Monitoring Devices → DMA Controller → Main Memory

Emergency Devices → Nested Priority Interrupt Controller → CPU

Normal Monitoring Devices → DMA Buffers → Main Memory

The emergency interrupt path is given the highest priority.

Working

1. Patient-monitoring devices continuously generate data.
2. Devices may operate at different sampling rates.
3. Normal monitoring data is collected in buffers.
4. DMA transfers continuous monitoring data directly into memory.
5. The CPU does not need to process every normal data transfer.
6. If an emergency condition occurs, the device generates a high-priority interrupt.
7. The nested interrupt controller immediately sends the emergency interrupt to the CPU.
8. The CPU temporarily suspends lower-priority processing.
9. The emergency event is handled first.
10. After emergency processing is completed, the CPU resumes the previous task.
11. DMA continues transferring normal monitoring data in the background.

Example

Suppose a patient-monitoring system is continuously collecting heart-rate and other vital measurements.

Normal monitoring:

Sensors → DMA → Memory

Emergency condition:

Emergency Sensor → High-Priority Interrupt → CPU → Immediate Processing

Thus, continuous monitoring does not unnecessarily interrupt the CPU, while an emergency event receives immediate attention.

Why Nested Interrupts Are Suitable

Nested interrupts allow a high-priority emergency event to interrupt a lower-priority task. This is important in healthcare systems because emergency events cannot wait for normal monitoring operations to finish.

Why DMA Is Suitable

Continuous monitoring produces a large amount of data. DMA transfers this data directly to memory and reduces CPU involvement. This allows the CPU to remain available for critical processing.

Conclusion

The proposed healthcare I/O architecture combines DMA for continuous monitoring data with nested, priority-based interrupts for emergency events. This ensures efficient CPU utilization while providing immediate response to critical patient conditions.

Overall Conclusion

The five proposed I/O architectures demonstrate how different I/O techniques can be selected according to system constraints.

- **Robotic arm:** Priority-based interrupts for critical sensors and DMA for diagnostic data.
- **Autonomous vehicle:** PCIe and DMA for high-bandwidth camera data with priority handling for safety sensors.
- **Industrial monitoring:** Interrupt coalescing for normal sensor data and immediate interrupts for critical faults.
- **High-speed network:** DMA-based packet transfer with configurable interrupt batching.
- **Healthcare monitoring:** DMA for continuous data and nested interrupts for emergency events.

The selection of an I/O mechanism depends on **latency, throughput, CPU utilization, data rate, device priority, and response-time requirements**. Priority interrupts are useful for time-critical events, while DMA is suitable for high-volume data transfer because it reduces CPU involvement. Interrupt coalescing further reduces processor overhead, while high-speed interfaces such as PCIe support large data rates.

Therefore, the proposed architectures satisfy the given constraints while improving **CPU efficiency, communication performance, interrupt response, and overall system reliability**.